

GUIDE TECHNIQUE · DÉPLOIEMENT GOOGLE CLOUD PLATFORM

Guide Technique GCP du projet MGVaovao

Destiné aux techniciens — développeurs · ML engineers · Cloud architects · DevOps. Pas-à-pas, service par service : du projet et des comptes de service à l'inférence GPU, au pipeline Vertex AI et au MLOps.

Tsantaniaina Rakotonjanahary Ambaorasoa Guillaume



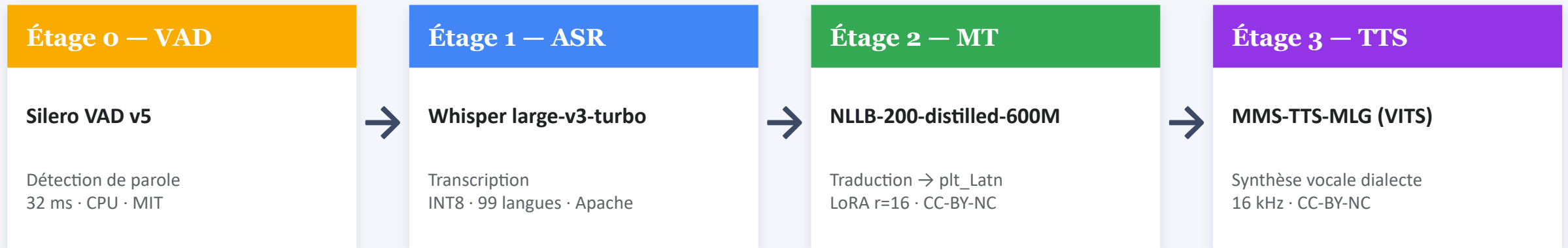
Préparé par : **Tsantaniaina Rakotonjanahary Ambaorasoa Guillaume**

Projet : mgvaovao-ia · Région : us-central1 · Traduction vocale temps réel anglais/français → malgache (dialectes)

Le projet MGVaovao

MGVaovao traduit automatiquement de l'audio en langue étrangère (vidéos, podcasts, parole en direct) vers du malgache parlé, spécifique au dialecte choisi — sans cliquer sur un bouton. L'utilisateur choisit la langue source et le dialecte cible ; le système détecte la parole, la transcrit, la traduit puis la synthétise.

LE PIPELINE EN CASCADE — 4 ÉTAGES INDÉPENDANTS



Tsantaniaina Rakotonjanahary Ambaorasoa Guillaume

POURQUOI EN CASCADE ? Modularité : chaque composant est remplaçable/affinable indépendamment. Fine-tuning ciblé du maillon le plus faible. Extension dialecte à bas coût (1 checkpoint TTS/dialecte). Empreinte totale ~6,2 Go sur GPU L4 (24 Go). Aucun système end-to-end S2S pour le malgache n'existait publiquement en 2026.

Dialectes & pile de modèles

DIALECTES MALGACHES SUPPORTÉS

Clé	Nom	Région	Locuteurs
plt_latn	Malagasy Officiel	Antananarivo	Référence
betsileo	Betsileo	Fianarantsoa	~1,5 M
betsimisaraka	Betsimisaraka	Toamasina	~1,5 M
sakalava	Sakalava	Mahajanga	~1 M
antandroy	Antandroy	Ambovombe	~800 K
merina	Merina	Imerina	~4 M

Langues sources : fr · en · de · es · it · pt (auto-détectées par Whisper)

PILE DE MODÈLES IA & VRAM (NVIDIA L4 24 GB)

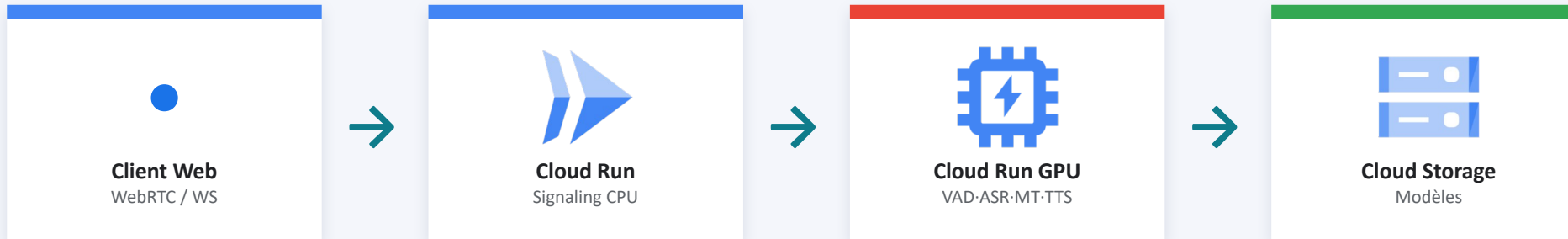
Étage	Modèle	VRAM	Fine-tune
VAD	Silero VAD v5	0 (CPU)	Non requis
ASR	Whisper turbo INT8	1,7 GB	Optionnel
MT	NLLB-200-600M INT8	0,7 GB	Oui (haut)
TTS	MMS-TTS-MLG ×6	3,0 GB	Oui /dialecte
—	Overhead CUDA	0,8 GB	—
—	TOTAL utilisé	6,2 GB	17,8 GB libre

À RETENIR 3 à 5 sessions WebRTC/WebSocket simultanées par instance. L4 Spot ~0,28 \$/h. NLLB et MMS-TTS sous CC-BY-NC 4.0 — usage ONG/recherche autorisé. Whisper Apache 2.0, Silero MIT.



Architecture GCP de production

① TEMPS RÉEL — INFÉRENCE



Tsantaniaina Rakotonjanahary Ambaorasoa Guillaume

② COLLECTE → ENTRAÎNEMENT → DÉPLOIEMENT



La chaîne ② réinjecte le modèle validé dans Cloud Run GPU (chaîne ①) via un déploiement Blue/Green.

TRANSVERSAUX



Artifact Registry



IAM



Logging



Monitoring



Compute Engine

Flux d'architecture & latence

LES 6 FLUX DE L'ARCHITECTURE

Flux	Chemin	Composants clés
2a	Connexion WebRTC	Client → Cloud Run CPU
2b	Pipeline d'inférence	Cloud Run CPU → Cloud Run GPU L4
2c	Stockage modèles	Cloud Run GPU ↔ Cloud Storage
2d	Déclencheur training	GCS → Pub/Sub → Vertex Pipeline
2e	Cycle de vie modèle	Vertex → Registry → Build → Deploy
2f	Monitoring de drift	Scheduler → Monitoring → Pub/Sub

LATENCE (L4, À CHAUD)

Étage	Latence
Silero VAD	1–50 ms
Whisper INT8 (3 s)	250–500 ms
NLLB-200 INT8	80–350 ms
MMS-TTS VITS	150–450 ms
WebRTC transport	50–150 ms
TOTAL (à chaud)	~700–1500 ms

Tsantaniaina Rakotonjanahary Ambaorasoa Guillaume

OPTIMISATIONS ACTIVES vad_filter=True (VAD + ASR en parallèle) · num_beams=2 · checkpoints TTS préchargés en VRAM · chunks de 32 ms · quantification INT8 (CTranslate2) : –50 % VRAM, ×2 vitesse.





Kozy — application de collecte de données

Kozy (dépôt Frontend) est l'application web qui alimente le projet en données. Elle collecte et cure des échantillons audio par dialecte et des jeux CSV de traduction, stockés sur Cloud Storage — la matière première du fine-tuning NLLB et TTS.

PILE TECHNIQUE

- Frontend : React 19 · Vite · Tailwind CSS
- Backend : Node.js · Express · TypeScript
- Auth : Google OAuth 2.0 (google-auth-library)
- Upload : Multer · CSV : Papa Parse · validation Zod

DONNÉES PRODUITES

```
# Bucket : gs://kozy-dataset
csv/      # source,target,
          #   src_lang,dialecte
audio/{dialect}/{fichier}.wav
```

Tsantaniaina Rakotonjanahary Ambaorasoa Guillaume

Source : dépôt Frontend (Kozy) — gcsService.ts · cloudbuild.yaml · Dockerfile · DEPLOY.md

COMPOSANTS GCP UTILISÉS



Cloud Storage

bucket kozy-dataset



Google OAuth 2.0

authentification



Cloud Run

hébergement · port 8080



Cloud Build

CI/CD



Artifact Registry

image kozy



Secret Manager

GOOGLE_CLIENT_ID



Kozy — déploiement & sécurité

PIPELINE CLOUD BUILD (cloudbuild.yaml)

```
# 1. build (VITE_GOOGLE_CLIENT_ID injecté)
docker build -t ../mgvaovao/kozy .
# 2. push -> Artifact Registry
# 3. deploy -> Cloud Run
gcloud run deploy kozy \
  --image ../mgvaovao/kozy \
  --region us-central1 --port 8080 \
  --min-instances 0 --max-instances 5 \
  --set-secrets=GOOGLE_CLIENT_ID=\
    GOOGLE_CLIENT_ID:latest
```

COMPTE DE SERVICE DÉDIÉ (MOINDRE PRIVILÈGE)

```
storage-kozy@PROJECT.iam.gserviceaccount.com
roles/storage.objectAdmin
roles/secretmanager.secretAccessor
```

SECRET MANAGER

Coffre-fort managé pour les secrets (clés, identifiants). Le `GOOGLE_CLIENT_ID` OAuth y est stocké puis injecté à l'exécution dans Cloud Run via `--set-secrets` — jamais en clair dans l'image ni le code.

- Versions de secrets + rotation
- Accès contrôlé par IAM (secretAccessor)
- Injecté comme variable d'environnement au runtime

SÉCURITÉ — BONNES PRATIQUES Identifiants via Application Default Credentials (ADC), aucune clé de SA embarquée. OAuth 2.0 pour authentifier les utilisateurs. Secrets dans Secret Manager. Compte de service dédié au strict minimum de rôles.

Tsantaniaina Rakotonjanahary Ambaorasoa Guillaume

01

PARTIE 1 — PRÉ-REQUIS & CONFIGURATION DU PROJET

Fondations GCP



Tsantaniaina Rakotonjanahary Ambaoraso Guillaume

Pré-requis

CÔTÉ POSTE DE TRAVAIL

- gcloud CLI installé et authentifié
- git
- Python 3.10+ (pour les scripts utilitaires)
- Docker (optionnel — le build se fait dans le cloud)

CÔTÉ GOOGLE CLOUD

- Un projet GCP avec la facturation activée
- Quota GPU L4 approuvé (Cloud Run + Vertex)
- 10 APIs à activer (slide suivante)
- Droits IAM pour créer des comptes de service

LES 10 APIs REQUISES

```
run.googleapis.com           # Cloud Run
artifactregistry.googleapis.com # Artifact Reg.
cloudbuild.googleapis.com     # Cloud Build
storage.googleapis.com        # Cloud Storage
aiplatform.googleapis.com     # Vertex AI
pubsub.googleapis.com         # Pub/Sub
cloudscheduler.googleapis.com # Scheduler
logging.googleapis.com        # Cloud Logging
monitoring.googleapis.com     # Cloud Monitoring
iam.googleapis.com            # IAM
```

Installer & configurer gcloud

1

Installer & s'authentifier

gcloud auth login + application-default login

2

Créer ou sélectionner le projet

projet mgvaovao-ia

3

Activer la facturation

Console → Billing → lier un compte

4

Noter le numéro de projet

nécessaire pour les bindings IAM

5

Enregistrer un profil gcloud nommé

évite de retaper projet/région

```
# 1. Authentification
gcloud auth login
gcloud auth application-default login

# 2-3. Projet + facturation (Console)
gcloud projects create mgvaovao-ia \
  --name="MGVaovao IA"
gcloud config set project mgvaovao-ia

# 4. Numéro de projet
gcloud projects describe mgvaovao-ia \
  --format="value(projectNumber)"
# -> 97374817504

# 5. Profil nommé
gcloud config configurations create mgvaovao
gcloud config set compute/region us-central1
gcloud config set compute/zone us-central1-a
```

Tsantaniaina Rakotonjanahary Ambaorasoa Guillaume

Activer les APIs

Activez les 10 services en une commande. L'opération prend 1 à 3 minutes. Chaque API doit être active avant d'utiliser le service correspondant.

```
gcloud services enable \  
  run.googleapis.com \  
  artifactregistry.googleapis.com \  
  cloudbuild.googleapis.com \  
  storage.googleapis.com \  
  aiplatform.googleapis.com \  
  pubsub.googleapis.com \  
  cloudscheduler.googleapis.com \  
  logging.googleapis.com \  
  monitoring.googleapis.com \  
  iam.googleapis.com \  
  --project=mgvaovao-ia
```

VÉRIFIER

```
gcloud services list \  
  --enabled \  
  --project=mgvaovao-ia \  
  | grep -E \  
  "run|build|aiplatform"
```

ASTUCE Réactiver une API déjà active est sans effet (idempotent). Sauter les étapes déjà faites lors d'une migration.

02

PARTIE 2 — CONCEPTS + CONFIGURATION PAS-À-PAS

Les services, un par un



Tsantaniaina Rakotonjanahary Ambaoraso Guillaume



IAM — concepts

Identity and Access Management contrôle « qui peut faire quoi, sur quelle ressource ». On accorde des RÔLES (ensembles de permissions) à des PRINCIPALS (utilisateurs ou comptes de service) via des bindings sur une ressource.

CONCEPTS CLÉS

- Principals : utilisateurs, groupes, comptes de service
- Rôles : prédéfinis, personnalisés, ou basiques
- Permission = service.resource.verb (ex. run.services.create)
- Binding = (principal + rôle) sur une ressource
- Moindre privilège + héritage par la hiérarchie

LES 2 COMPTES DE SERVICE DU PROJET

Compte de service	Rôles
cloudbuild-runner	run.admin · iam.serviceAccountUser · artifactregistry.writer · storage.objectAdmin
N-compute@developer (défaut)	storage.objectAdmin · artifactregistry.reader

Tsantaniaina Rakotonjanahary Ambaorasoa Guillaume

POURQUOI UN COMPTE DÉDIÉ ? Cloud Build doit déployer sur Cloud Run et pousser des images : on lui crée un compte de service dédié avec le strict minimum de rôles, plutôt que d'utiliser un compte trop permissif. C'est l'application du principe de moindre privilège.



IAM — créer le service account

1

Créer le compte de service

cloudbuild-runner

2

Cloud Run Admin

créer / mettre à jour les services

3

Service Account User

exécuter Cloud Run en tant que SA

4

Artifact Registry Writer

pousser les images Docker

5

Storage Object Admin

lire/écrire les checkpoints

```
PROJECT_NUMBER=97374817504
SA=cloudbuild-runner@mgvaovao-ia\
.iam.gserviceaccount.com

# 1. Créer le compte de service
gcloud iam service-accounts create \
  cloudbuild-runner \
  --display-name="Cloud Build deploy"

# 2-5. Accorder les rôles
for R in run.admin \
  iam.serviceAccountUser \
  artifactregistry.writer \
  storage.objectAdmin; do
  gcloud projects \
    add-iam-policy-binding mgvaovao-ia \
    --member="serviceAccount:$SA" \
    --role="roles/$R"
done
```

Tsantaniaina Rakotonjanahary Ambaorasoa Guillaume



IAM — comptes & vérification

Deux comptes de service gérés par Google reçoivent aussi des rôles : celui utilisé par « gcloud builds submit » et le compte Compute par défaut utilisé par les jobs Vertex AI.

SA PAR DÉFAUT DE CLOUD BUILD

```
# Utilisé par: gcloud builds submit
SA=$PROJECT_NUMBER@cloudbuild\
.gserviceaccount.com

gcloud projects add-iam-policy-binding \
  mgvaovao-ia --member="serviceAccount:$SA" \
  --role="roles/run.admin"

gcloud projects add-iam-policy-binding \
  mgvaovao-ia --member="serviceAccount:$SA" \
  --role="roles/iam.serviceAccountUser"
```

SA COMPUTE (VERTEX AI)

```
# Utilisé par les jobs Vertex AI
SA=$PROJECT_NUMBER-compute@\
developer.gserviceaccount.com

# Accès GCS (checkpoints/datasets)
--role="roles/storage.objectAdmin"

# Lecture des images Docker
--role="roles/artifactregistry.reader"
```

VÉRIFIER gcloud iam service-accounts list --project=mgvaovao-ia → doit lister cloudbuild-runner et le compte compute par défaut.



Artifact Registry — concepts

Registre managé pour packages et images de conteneurs (successeur de Container Registry). Les images Docker du projet y sont stockées, puis tirées par Cloud Run (inférence) et Vertex AI (training).

CONCEPTS CLÉS

- Dépôts régionaux multi-formats (Docker, Python, npm...)
- Contrôle d'accès via IAM (writer / reader)
- Analyse de vulnérabilités (Artifact Analysis)
- Cible des builds, source des déploiements

LES 2 IMAGES DU PROJET

```
# Dépôt : us-central1 - format docker

# Image d'inférence (Cloud Run GPU)
.../mgvaovao/inference:latest
.../mgvaovao/inference:{SHORT_SHA}

# Image CUDA (training Vertex)
.../mgvaovao/mgvaovao:cuda-latest
```

CONVENTION DE TAG Chaque commit produit un tag immuable {SHORT_SHA} + un tag mobile latest. Le déploiement utilise le tag immuable pour la traçabilité et le rollback.



Artifact Registry — créer le dépôt

1

Créer le dépôt Docker

nom mgvaovao · région us-central1

2

Vérifier la création

repositories list

3

Authentifier Docker (si push local)

configure-docker

```
# 1. Créer le dépôt
gcloud artifacts repositories create \
  mgvaovao \
  --repository-format=docker \
  --location=us-central1 \
  --description="MGVaovao images" \
  --project=mgvaovao-ia

# 2. Vérifier
gcloud artifacts repositories list \
  --project=mgvaovao-ia

# 3. Auth Docker (push local)
gcloud auth configure-docker \
  us-central1-docker.pkg.dev
```



Cloud Storage — concepts

Stockage objet scalable. Conserve les checkpoints fine-tunés (tirés par Cloud Run à froid) et les datasets d'entraînement/évaluation. Backend natif de Vertex AI.

CONCEPTS CLÉS

- Objets rangés dans des buckets (régional/multi-régional)
- Classes Standard / Nearline / Coldline / Archive
- Versioning d'objets (récupération, audit)
- Accès fin via IAM, chiffrement par défaut

LES 2 BUCKETS DU PROJET

Bucket	Rôle
mgvaovao-ia-checkpoints	Checkpoints modèles (tirés à froid)
mgvaovao-ia_cloudbuild	Archives source Cloud Build (auto)

Tsantaniaina Rakotonjanahary Ambaoraso Guillaume

AU DÉMARRAGE À FROID Cloud Run exécute pull_checkpoints.py qui télécharge uniquement les checkpoints absents en local. Si le bucket est vide, les modèles de base (intégrés à l'image) sont utilisés.





Cloud Storage — buckets & checkpoints

```
# 1. Créer le bucket de checkpoints
gcloud storage buckets create \
  gs://mgvaovao-ia-checkpoints \
  --location=us-central1 \
  --project=mgvaovao-ia
# (le bucket cloudbuild est auto-crée)
```

ARBORESCENCE GCS

```
gs://mgvaovao-ia-checkpoints/
  tts_{dialect}/final/      # VITS
    config.json model.safetensors
  nllb_{dialect}/final/    # LoRA
    adapter_config.json
    adapter_model.safetensors
```

UPLOADER UN CHECKPOINT FINE-TUNÉ

```
# Script du projet
python scripts/push_checkpoint.py \
  tts betsileo \
  ./checkpoints/tts_betsileo/final

# Ou directement avec gsutil
gsutil -m cp -r \
  ./checkpoints/tts_betsileo/final \
  gs://mgvaovao-ia-checkpoints/tts_betsileo/
```

PRENDRE EN COMPTE LE NOUVEAU MODÈLE Les checkpoints ne sont tirés qu'au démarrage à froid. Forcer un redémarrage :

```
gcloud run services update mgvaovao-inference --region=us-central1
```



Compute Engine — concepts

IaaS de Google Cloud : machines virtuelles configurables. C'est le socle des accélérateurs GPU utilisés par Vertex AI (training) et Cloud Run (inférence). On n'y déploie pas de VM manuellement ici, mais on en consomme les GPU et on demande le quota.

CONCEPTS CLÉS

- Familles de machines (n1, n2, accelerator-optimized...)
- Accélérateurs GPU : NVIDIA T4 (training), L4 (inférence)
- Persistent Disk / Local SSD · images Linux/Windows
- Spot VMs : -60 % à -91 % vs on-demand
- Régions & zones (us-central1 / us-central1-a)

LES GPU DU PROJET

Usage	Machine + GPU
Training (Vertex)	n1-standard-8 + NVIDIA_TESLA_T4
Inférence (Run)	g2-standard-8 + NVIDIA L4 24 GB
Coût L4 Spot	~0,28 \$/h

Tsantaniaina Rakotonjanahary Ambaoraso Guillaume

QUOTA GPU Cloud Run GPU L4 est GA depuis juin 2025. Le quota GPU doit être approuvé pour votre projet/région avant déploiement : Console → IAM & Admin → Quotas → filtrer « L4 ».





Cloud Build — concepts

Service qui exécute vos builds sur Google Cloud. Pour MGVaovao : il construit l'image d'inférence (les 4 modèles sont intégrés à l'image, ~8 Go), la pousse dans Artifact Registry, puis déploie sur Cloud Run GPU — sans interruption (Blue/Green).

CONCEPTS CLÉS

- Config déclarative cloudbuild-inference.yaml (étapes)
- Chaque étape = un conteneur (builder)
- Substitutions de variables (\$SHORT_SHA, \$PROJECT_ID)
- Déclencheurs (push GitHub) + cache de couches Docker

TEMPS DE BUILD

Build	Durée
1er build (téléch. modèles)	~45–60 min
Buids suivants (cache)	~5–10 min
Taille de l'image	~8 GB

Tsantaniaina Rakotonjanahary Ambaoraso Guillaume

3 FICHIERS IMPLIQUÉS Dockerfile.cloudrun (image multi-stage) · cloudbuild-inference.yaml (pipeline build→push→deploy) · scripts/download_models.py (télécharge les 4 modèles au build).



Cloud Build — le pipeline étape par étape

1

build-inference

`docker build -f Dockerfile.cloudrun, tags {SHA}+latest, --cache-from`

2

push-sha

`pousse inference:{SHORT_SHA}`

3

push-latest

`pousse inference:latest`

4

deploy

`gcloud run deploy avec GPU L4 (tag immuable)`

```
# Étape 1 – build
docker build -f Dockerfile.cloudrun \
  --tag inference:{SHORT_SHA} \
  --tag inference:latest \
  --cache-from inference:latest

# Étape 2-3 – push vers Artifact Registry
push inference:{SHORT_SHA}
push inference:latest

# Étape 4 – déploiement Cloud Run
gcloud run deploy mgvaovao-inference \
  --image inference:{SHORT_SHA} \
  --gpu 1 --gpu-type nvidia-l4 \
  --cpu 8 --memory 32Gi \
  --min-instances 0 --max-instances 2
```

Tsantaniaina Rakotonjanahary Ambaorasoa Guillaume



Cloud Build — lancer & suivre

LANCER UN BUILD MANUEL

```
# Linux / macOS / WSL
gcloud builds submit \
  --config=cloudbuild-inference.yaml \
  --project=mgvaovao-ia \
  --substitutions=SHORT_SHA=$(git \
    rev-parse --short HEAD) .
```

```
# PowerShell (Windows)
$SHA = git rev-parse --short HEAD
gcloud builds submit `
  --config=cloudbuild-inference.yaml `
  "--substitutions=SHORT_SHA=$SHA" .
```

SUIVRE UN BUILD

```
gcloud builds list --limit=5 \
  --project=mgvaovao-ia

gcloud builds log BUILD_ID \
  --project=mgvaovao-ia --stream
```

PIÈGE — SHORT_SHA VIDE \$SHORT_SHA n'est rempli automatiquement que par un déclencheur. En build manuel il est vide → erreur « invalid image name ». Toujours le passer explicitement (sur PowerShell, via une variable intermédiaire).

Tsantaniaina Rakotonjanahary Ambaorasoa Guillaume



Cloud Run — concepts

Plateforme serverless pour conteneurs, sans gérer de serveurs. Deux services dans MGVaovao : un service CPU léger (signaling WebRTC, ~2 \$/mois) et un service GPU L4 (pipeline d'inférence complet, ~33 \$/mois). Les deux passent à l'échelle zéro.

CONCEPTS CLÉS

- Conteneurs serverless, autoscaling à la demande
- Scale-to-zero (0 € à l'idle) — même avec GPU
- Facturation à l'usage · HTTP/2, gRPC, WebSockets
- Cloud Run écoute toujours sur le port 8080

LES 2 SERVICES

Service	Rôle / coût
Cloud Run CPU	Signaling WebRTC · ~2 \$/mois
Cloud Run GPU L4	VAD+ASR+MT+TTS · ~33 \$/mois

Tsantaniaina Rakotonjanahary Ambaoraso Guillaume

GPU SANS PILOTE À INSTALLER Les pilotes NVIDIA sont montés automatiquement (/usr/local/nvidia). L4 exige au minimum 8 vCPU et 32 Gio de RAM sur Cloud Run, et 1 seul GPU par instance.



Cloud Run GPU — la commande de déploiement

```
gcloud run deploy mgvaovao-inference \
  --image=.../inference:{SHA} \
  --region=us-central1 \
  --platform=managed \
  --gpu=1 \
  --gpu-type=nvidia-l4 \
  --cpu=8 \
  --memory=32Gi \
  --no-cpu-throttling \
  --no-gpu-zonal-redundancy \
  --concurrency=1 \
  --min-instances=0 \
  --max-instances=2 \
  --timeout=300 \
  --port=8080 \
  --set-env-vars=MGVAOVAO_DEVICE=cuda \
  --allow-unauthenticated
```

Flag	Pourquoi
--gpu-type nvidia-l4	24 GB VRAM, GA 2025
--cpu 8 / --memory 32Gi	minimum L4 sur Run
--no-cpu-throttling	CPU pour scheduling GPU
--concurrency 1	1 session WS / instance
--min/--max 0 / 2	scale-to-zero, max 2
--timeout 300	sessions WS longues
--port 8080	port fixe Cloud Run

Tsantaniaina Rakotonjanahary Ambaoraso Guillaume





Cloud Run — vérifier & démarrage à froid

VÉRIFIER LE DÉPLOIEMENT

```
# URL du service
gcloud run services describe \
  mgvaovao-inference \
  --region=us-central1 \
  --format="value(status.url)"

# Santé / disponibilité
curl $URL/health # {"status":"ok"}
curl $URL/ready  # 503 puis 200
# UI temps réel : $URL/live
```

DÉMARRAGE À FROID (60–120 s)

- 1 Conteneur + CUDA**
init driver (~10–15 s)
- 2 Port 8080 ouvert**
/health répond aussitôt
- 3 Chargement modèles**
pull GCS + VAD/ASR/MT/TTS
- 4 /ready → 200**
l'UI se débloque, micro auto

ANTI COLD-START min-instances=0 éteint l'instance après ~15 min d'inactivité. Cloud Scheduler envoie un ping toutes les 15 min pendant les heures actives pour garder une instance chaude ; scale-to-zero la nuit/week-end.



Vertex AI — concepts

Plateforme ML unifiée. Pour MGVaovao : exécute le fine-tuning NLLB (LoRA) et TTS (VITS) sur GPU cloud, orchestre le cycle via un pipeline Kubeflow, versionne les modèles (Model Registry) et surveille le drift (Model Monitoring).

COMPOSANTS UTILISÉS

- Custom Training Jobs : votre code sur T4/L4
- Pipelines : DAG Kubeflow (6 étapes) — \$0,03/run
- Model Registry : candidate → staging → prod (gratuit)
- Model Monitoring : drift chrF++ & UTMOS (~3 \$/mois)
- Experiments : comparaison des runs (gratuit)

COÛT DU FINE-TUNING (T4 SPOT)

Job	Durée	Coût
NLLB LoRA (5 ép.)	4–6 h	~1,7–2,5 \$
TTS VITS (50 ép.)	2–3 h	~0,84–1,3 \$

Tsantaniaina Rakotonjanahary Ambaorasoa Guillaume

SPOT + CHECKPOINTS Ajouter `--scheduling=spot` (~60 % moins cher, mais préemptible). Toujours sauvegarder un checkpoint vers GCS toutes les 30 min ; `pull_checkpoints.py` gère la reprise.



Vertex AI — lancer un job de training

1

Donner accès GCS au SA compute

roles/storage.objectAdmin

2

Donner accès Artifact Registry

roles/artifactregistry.reader

3

Lancer le custom-job

machine + GPU + image + module

4

Suivre les logs

stream-logs / Console

```
# 3. Fine-tuning NLLB (dialecte betsileo)
gcloud ai custom-jobs create \
  --region=us-central1 \
  --project=mgvaovao-ia \
  --display-name="nllb-ft-betsileo" \
  --worker-pool-spec="
    machine-type=n1-standard-8,
    replica-count=1,
    accelerator-type=NVIDIA_TESLA_T4,
    accelerator-count=1,
    executor-image-uri=
      .../mgvaovao:cuda-latest,
    local-package-path=.,
    python-module=training.train_nllb" \
  --args="--dialect=betsileo"

# 4. Suivre les logs
gcloud ai custom-jobs stream-logs JOB_ID \
  --region=us-central1
```

Tsantaniaina Rakotonjanahary Ambaoraso Guillaume



Vertex AI Pipeline — le DAG Kubeflow



Déclencheur : dataset validé dans GCS → Pub/Sub (topic dataset-ready) → le pipeline démarre

DÉCISION APRÈS ÉVALUATION

- Régression détectée → ROLLBACK auto + alerte e-mail
- Pas de régression → register puis deploy (Blue/Green)
- Tous les runs comparés dans Vertex AI Experiments

COÛT & SÉCURITÉ Pipeline ~0,03 \$/run. Le déploiement est Blue/Green (zéro downtime) avec rollback automatique en cas de régression chrF++/UTMOS. Model Registry et Experiments sont gratuits.



Pub/Sub — concepts & rôle

Service de messagerie asynchrone qui découple producteurs et consommateurs. Dans MGVaovao, c'est le bus d'événements qui relie l'arrivée d'un dataset (ou la détection d'un drift) au démarrage du pipeline d'entraînement Vertex AI.

CONCEPTS CLÉS

- Topics & subscriptions (publishers / subscribers)
- Livraison push ou pull, au moins une fois
- Découplage & scalabilité, latence ~100 ms
- Topic du projet : dataset-ready

LES 2 DÉCLENCHEURS

```
# Flux 2d — nouveau dataset  
GCS (upload) -> Pub/Sub  
-> Vertex AI Pipeline  
  
# Flux 2f — drift détecté  
Scheduler -> Monitoring  
-> Pub/Sub -> re-fine-tuning
```

Tsantaniaina Rakotonjanahary Ambaorasoa Guillaume

POURQUOI UN BUS D'ÉVÉNEMENTS ? Le producteur (GCS / Monitoring) n'a pas besoin de connaître le consommateur (Vertex). On peut ajouter de nouveaux abonnés (alertes, logs) sans toucher au producteur — architecture découplée et robuste.



Cloud Logging — concepts & debug

Gestion de logs temps réel : stockage, recherche, analyse. Les builds (CLOUD_LOGGING_ONLY) et les services Cloud Run y envoient leurs logs, consultables en CLI ou dans Logs Explorer.

LIRE LES LOGS CLOUD RUN

```
gcloud logging read \  
"resource.type=cloud_run_revision \  
AND resource.labels.service_name=\  
mgvaovao-inference \  
AND severity>=ERROR" \  
--project=mgvaovao-ia \  
--limit=20 --freshness=1h
```

MOTIFS DE LOG À CONNAÎTRE

Ligne de log	Sens
Model loading started...	cold start, port ouvert
All pipelines loaded	/ready = 200
WS open — dialect=...	session WebSocket
VAD state=speaking	parole détectée
KeyError: 'bytes'	BUG (voir fixes)

CONSOLE Pour explorer visuellement : console.cloud.google.com/run → service → onglet Logs. Filtres par sévérité, requêtes sauvegardées, et trace associée aux entrées.



Cloud Monitoring — supervision & drift

Observabilité : métriques, dashboards, uptime checks et politiques d'alertes. Combiné à Vertex AI Monitoring, il surveille la santé du service ET la qualité des modèles (drift) en production.

SEUILS DE DRIFT & ACTIONS AUTOMATIQUES

Métrique	Seuil d'alerte	Action automatique
chrF++ (200 phrases de référence)	Baisse > 2 points	Re-fine-tuning NLLB via Pub/Sub
UTMOS (50 phrases audio)	Baisse > 0,3	Re-fine-tuning TTS via Pub/Sub
Taux d'erreur WebRTC	> 5 %	Alerte e-mail + investigation infra
Latence P95 du pipeline	> 2000 ms	Alerte e-mail + Cloud Monitoring

Tsantaniaina Rakotonjanahary Ambaoraso Guillaume

PRÉ-CHAUFFAGE & SUIVI Cloud Scheduler ping Cloud Run GPU toutes les 15 min (anti cold-start). Évaluation de drift hebdomadaire. Tous les runs d'entraînement sont tracés dans Vertex AI Experiments (gratuit).



03

PARTIE 3 — MLOPS, DÉPANNAGE, COÛTS & MIGRATION

Cycle de vie & exploitation





GitHub → Cloud Build (auto-trigger)

1

Connecter le dépôt GitHub

Console Cloud Build → Connect → autoriser

2

Créer le déclencheur

branche ^main\$ · cloudbuild-inference.yaml

3

Vérifier

builds triggers list

```
gcloud builds triggers create github \
  --project=mgvaovao-ia \
  --region=global \
  --name="deploy-inference-on-push" \
  --repo-owner=mgvaovao-Mdn \
  --repo-name=ml \
  --branch-pattern="^main$" \
  --build-config=\
    cloudbuild-inference.yaml
```

AVEC LE DÉCLENCHEUR Chaque git push origin main : build (cache ~5–10 min) → push Artifact Registry → deploy Cloud Run GPU. \$SHORT_SHA est rempli automatiquement (pas besoin de --substitutions). Sans déclencheur : toujours passer SHORT_SHA manuellement.

Problèmes connus & correctifs

Bug 1 — KeyError: 'bytes' à la déconnexion

```
# stream.py – frame disconnect sans clé bytes
if msg.get("type")=="websocket.disconnect":
    break
raw = msg.get("bytes") or b""
```

Bug 2 — Silero VAD ne se déclenche jamais

```
# config.py – seuil trop strict
vad_threshold = 0.3 # au lieu de 0.5
vad_stream_min_silence_ms = 400 # au lieu de 500
```

Bug 3 — SHORT_SHA vide (build manuel)

```
# image: → tag vide → invalid image name
--substitutions=SHORT_SHA=$(git \
rev-parse --short HEAD)
```

Tsantaniaina Rakotonjanahary Ambaorasoa Guillaume

Variables & migration

VARIABLES CLÉS DU PROJET

Variable	Valeur
GCP_PROJECT_ID	mgvaovao-ia
GCP_PROJECT_NUMBER	97374817504
GCP_REGION / ZONE	us-central1 / -a
AR_REPO	mgvaovao
GCS_BUCKET_MODELS	mgvaovao-ia-checkpoints
CLOUDRUN_SERVICE	mgvaovao-inference

MIGRER VERS UN NOUVEAU PROJET

- 1 Nouveau projet + facturation
- 2 Activer les APIs (§18.1)
- 3 Artifact Registry + buckets GCS
- 4 Recréer SA + rôles IAM
- 5 Transférer les checkpoints (gsutil)
- 6 1er build (~45–60 min) + vérifier

Tsantaniaina Rakotonjanahary Ambaoraso Guillaume

Coûts

RÉPARTITION MENSUELLE

Poste	Coût/mois
Cloud Run GPU L4 (50 h)	33,60 \$
Vertex AI Monitoring	3,00 \$
Vertex Custom Training (2/mois)	2,24 \$
Cloud Run CPU (signaling)	2,00 \$
Cloud Storage (50 GB)	1,00 \$
Artifact Registry	0,50 \$
Pipelines · Registry · Build	~0,12 \$
TOTAL	~42 \$/mois

OPTIMISATIONS ACTIVES

- Scale-to-zero (Cloud Run) : 0 € hors heures actives
- Spot instances (fine-tuning) : -60 % à -91 %
- INT8 (CTranslate2) : -50 % VRAM, ×2 vitesse
- Kaggle T4 ×2 gratuit : 30 h/semaine
- LoRA (vs full FT) : fraction des paramètres

ÉVOLUTION PAR PHASE Bêta < 100 u/j : ~42 \$. Croissance 100–500 u/j : ~100 \$. Production > 500 u/j : ~500 \$+ → migrer vers un Vertex AI Endpoint.



Feuille de route

Phase	Statut	Livrables clés
1 — Baseline	✔ Fait	Pipeline VAD→ASR→NLLB→TTS · WebSocket · UI temps réel · images Docker
2 — Cloud Deploy	🔄 En cours (Q2 2026)	Cloud Run GPU L4 · buckets GCS · 1er pipeline Vertex · Model Registry
3 — Fine-Tuning v1	🕒 Q2–Q3 2026	3 000 paires FR/EN↔MG · NLLB LoRA · 200 phrases audio · MMS-TTS Officiel
4 — Multi-dialecte	🕒 Q3–Q4 2026	3 dialectes ×200 phrases · drift monitoring live · app Flutter v1
5 — Scale	🕒 2027	API REST publique · 6 dialectes · NLLB 1.3B · migration Vertex Endpoint

Tsantaniaina Rakotonjanahary Ambaorasoa Guillaume



Pour aller plus loin



Cloud Run cloud.google.com/run/docs



Cloud Run GPU cloud.google.com/run/docs/configuring/services/gpu



Cloud Storage cloud.google.com/storage/docs



Compute Engine cloud.google.com/compute/docs



Vertex AI cloud.google.com/vertex-ai/docs



Cloud Build cloud.google.com/build/docs



Artifact Registry cloud.google.com/artifact-registry/docs



Pub/Sub cloud.google.com/pubsub/docs



Cloud Logging cloud.google.com/logging/docs



Cloud Monitoring cloud.google.com/monitoring/docs



IAM cloud.google.com/iam/docs



Secret Manager cloud.google.com/secret-manager/docs

Réf. README §24 : Cloud Run GPU GA (juin 2025) · Vertex AI pricing · NLLB-200 · MMS-TTS-MLG · Silero VAD · CTranslate2 · aiortc

Tsantaniaina Rakotonjanahary Ambaoraso Guillaume